

Ruminations On Cantor's Diagonal Method

Thomas Walker Lynch

2025-02-03

1 Survey

In the year 1891, Georg Cantor gave us an algorithm for constructing an exception to a proposed exhaustive enumeration of numbers.

[Reference some Cantor works, Kleene book.] [Reference Papadimitriou on the grids] [Reference the Cambridge guy on digit direction] https://github.com/Thomas-Walker-Lynch/Ariadne/tree/examples_1.0 for the Index Tree

2 Cantor's Algorithm Applied to an Enumerable Set

Cantor diagonal method is an algorithm that can be said to have three phases: initial setup, iteration, and finalization. The goal of the algorithm is to construct the 'exception sequence', which at the end is to have a symbol sequence not found in the table. The exception sequence is built up symbol by symbol, step by step, with each step resulting in a longer prefix.

An important function for the algorithm is the 'representation function'. It maps numbers to symbol sequences. Such a symbol sequence is called the number's 'representation'. The inverse of the representation function maps the symbol sequence back to a number. We have no way of writing numbers, but we can write represents, so it follows that any visual method, such as Cantor's diagonalization method, will be based on representations.

A function we will call the 'symbol altering function' also plays an important role. This function is given any one symbol chosen from the alphabet of symbols used by the representation function, and then returns a different symbol from said alphabet. For example, when using binary Arabic notation, the alphabet consists of two symbols, 0 and 1. The symbol altering function used need not be the sole unique function possible for accomplishing this feat.

The initial phase requires using a representation function to construct a table such that each number from a proposed enumeration is represented on a row, with its symbol sequence running across the columns. For a not too long enumeration the table can be explicitly constructed; otherwise the table must be described using mathematical abstraction. A table can be set up in a manner such that the exception sequence will begin with a predefined prefix, but in this section, the table will be setup such that the initial exception prefix will be empty.

Starting the iteration phase at step 0, the algorithm reads the first symbol on the diagonal, i.e. the one located at row 0 column 0. This symbol is given to the symbol altering function, which returns a different symbol. This new symbol is appended to the exception sequence to create the first exception sequence prefix, which will be of length 1. Upon completion of step 0, the exception sequence prefix will differ from the row 0 sequence due to the work of the symbol altering function in changing the first symbol.

Continuing in this manner, at step n , the row n column n symbol will be given to the symbol altering function, and then the symbol returned will be appended to the exception sequence prefix, so as to make the prefix longer by one symbol. It follows that for any value n , this algorithm will cause the constructed exception prefix to differ from all the sequences found in rows 0 through row n .

By inductive reasoning it is then concluded that this algorithm will produce a exception sequence not found in the table.

In the final phase, the inverse representation function is applied to the exceptional sequence, and it returns the exception number. When the algorithm is run to answer existence questions such as: "Does a number exist that is not in the table?", it is sufficient to show the diagonal can be constructed, so there is no need for this final phase.

This algorithm is called the 'Diagonal Method' because all the row n column n symbols are of course found along the primary diagonal of the table that was created as part of initialization phase. This algorithm will work only with representation functions that will cause the constructed exception sequence to correspond to a number which is distinct from the numbers represented by the rows. Any representation function that creates a one to one correspondence between symbol sequences and numbers will suffice, but one to one correspondence is not necessary. The only necessary requirement is that the representation of the distinct exception sequence corresponds to a distinct number from the rows, though this criteria approaches being circular.

Cantor's algorithm is often introduced to demonstration that an enumeration of numbers with fraction digits, say of the form $1.xxxxxx\dots$ fails to be exhaustive, thus proving that the set of real numbers between 1 and 2 is uncountable. However, a more structured presentation of the method would first apply Cantor's algorithm to a sequence of Natural Numbers so as to gain an intuition as to what it looks

2 CANTOR'S ALGORITHM APPLIED TO AN ENUMERABLE SET

like when fails to produce an exception. Hence in this section we will examine an exhaustive enumeration of the binary Arabic representation symbol sequences for the **Natural Numbers**, and demonstrate that the construction of a sequence from the diagonal fails to be exceptional. The Arabic representation for Natural Numbers creates a one to one mapping between Natural Numbers and symbol sequences, so it meets the requirements for a representation function. It is also very close to the Peano definition of the Natural Numbers.

The table below illustrates an enumeration of the Natural Numbers in binary form. Zero is included in this definition so that arithmetic, should we need to use it, will have an additive identity.

			0	0	0	0	0	0
			0	0	0	0	0	1
			0	0	0	0	1	0
			0	0	0	0	1	1
			0	0	0	1	0	0
		0	0	0	1	0	1	
	0	0	0	1	1	1	0	
0	0	0	0	1	1	1	1	
			0	1	0	0	0	
			0	1	0	0	1	
			0	1	0	1	0	
			0	1	0	1	1	
			0	1	1	0	0	
			0	1	1	0	1	
			0	1	1	1	0	
			0	1	1	1	1	
								.
								.
								.

Table 1: Step by Step, Diagonal Prefix

The number of symbols used to represent a number in Arabic representation, i.e. the length of a number, is logarithmically proportional to the value of the number being represented. This can be seen in the table above by noting the location of the leftmost bit in each row of the table, and how it only slowly occupies a more leftward digit position when walking down the table. In contrast, the diagonal grows in length linearly when stepping down the table. Consequently, once the diagonal departs from the length of a represented number, it will be forevermore in the land of zero symbols.

For a binary string, the only function that always alters a bit, is the bit flip function. When given a 0 it returns a 1, and when given a 1 it returns a 0. If a number

representation has a 0 in bit position i , then flipping that bit will add 2^i to the number. Conversely, if a number representation has a 1 in bit position i , flipping the bit will subtract 2^i . In either case, the resulting number will be different than the given number, and will be greater than or equal to zero.

Thus, it seems that we have all the prerequisites necessary for constructing an exception number, as we have an enumeration of the numbers, digit by digit, in a table. We have a definition for the diagonal, and we have a function that produces a number that is unique to the table discovered thus far, as we walk down it step by step. It would seem then, by induction, we should be able to produce an exception to the enumeration of Natural Numbers.

The iteration phase begins at step 0 while focusing on row 0 column 0, where the table entry is found to be 0. This bit is flipped and 1 is appended to the currently empty exception number. As Cantor promised, this makes the exception number prefix different from the number in row zero; however, this exception number prefix is identical to the number in the next row, row 1. Stepping down to row 1, we rectify this problem by flipping the row 1 column 1 bit, and appending that to the exception number, yielding an exception number prefix of 11. The exception prefix is now indeed different from the row 0 number, and different from the row 1 number, but alas, it is now identical to the row 3 number. So as we proceed step by step we find the following values:

Step/Row	Exception Prefix	Found On	Distance
0	1	1	1
1	11	3	2
2	111	7	5
3	1111	15	12
4	11111	31	27

Table 2: Step-by-step exception number construction

The exception number prefix after step n will correspond to a value of $2^{(n+1)} - 1$. This symbol sequence will always be found $2^{(n+1)} - n - 1$ rows further down the table. Although the jump grows exponentially in size with each step, it will still always be in the table. Hence, the reason we can not apply the inductive proof as prescribed by the Cantor's diagonalization method is that at no step is the exception number not further down in the table.

It follows that we need to modify our definition of success in finding an exception. It is not sufficient that the exception prefix at step n is distinct from the rows 0 through n . Rather we must also assure that exception prefix is not always found further down in the table. Without this second requirement we have a found an inconsistency in the Peano construction of the Natural Numbers, which is the very definition of Natural Number.

It is interesting that if we ever stop enumerating the natural numbers at some bounding number, that exception numbers can be constructed from diagonals that will not be found in the table. Though this isn't so amazing, as incrementing the last entry in the table would accomplishing the same thing.

An ever growing string of 1s is an interesting number. The exception growing without bound is not unique to here, as the length of the rows in the table are also growing without bound. If we were to not speak of unbounded numbers, then we would also have to dismiss the diagonal method as unworkable due to the unbounded growth in row lengths.

In the binary Arabic notation for Natural Numbers, the bit weights are 2^j where j is the sequence index for the bit, starting from zero. If we instead use a negative power in the weights, 2^{-j} , without changing the table, the representations will be for the numbers between 0 and 1. And if we want a fraction in normal form, we can prepend a 1 to both the row sequences and the initial exception sequence, and without changing the table, numbers will range from 1 to 2. However, if we accept that an unbounded sequence of one symbols, $.11111\dots$ is equivalent to a carry, $1.00\dots$ then the Arabic representation for fractions fails to place symbol sequences into one to one correspondence with numbers. Thus applying Cantor's method to fractions requires an additional reasoning step that explains the effect, if any, of this recurrent wrinkle in the representation function mapping. We will come back to this in a later section.

3 The Index Tree Iterator

An Index Tree is an infinite graph. Each node has an unbounded number of children, and there is an unbounded number of levels in the tree. Each node has a label that consists of a sequence of Natural Numbers. The root node is an empty sequence. The unbounded number of children of the root node each has as a label a sequence with a single Natural Number. The next level down, the node sequences are two numbers long. Etc. An Index Tree iterator is an algorithm for traversing the Index Tree.

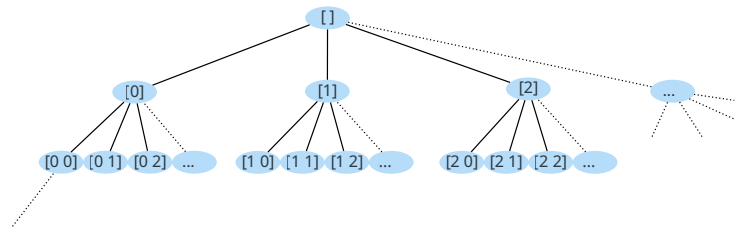


Figure 1: Example figure

The sequence of numbers in a node label can be read as the directions starting from the root node leading to the node in the tree. Hence, we know that each node label is distinct. Furthermore, an increment of one finds a neighbor node, so there are no sequences that do not correspond to nodes.

Index Tree nodes can not be enumerated with a depth first traversal, because the traversal would descend, never reach the bottom, and thus never backtrack nor step in the breadth direction, i.e. traversal would never not step across to a sibling node.

The tree nodes can not be enumerated using a breadth first traversal, because the traversal would walk indefinitely across one of the infinite sequences of siblings and then never descend nor step across to another parent node.

3.1 Grid

We see a similar problem with trying to traverse a two dimensional grid of nodes. Starting at the origin if traversal goes along the x -axis the traversal will never enumerate nodes going up the y -axis. Similarly a traversal that begins by ascending the y -axis will never make progress along the x -axis.

We know that having two unbounded free variables does not preclude the enumeration of all pair combinations, because diagonals or shells of the grid can be enumerated. The following function enumerates along diagonals. Where n is enumerated from 0, 1, 2,

$$n = x + y$$

Accordingly, when $n = 0$, x and y must be 0. When n is a non-zero number, x will range from 0 to n , while y takes on the value $n - x$ at each step. Due to both the diagonal index and the shell having to be enumerated, the enumeration will take t^2 number of steps, where t is the step counter. Each step will visit a single pair. And it must be this way because the area of the triangle that results from adding multiple diagonals is size $1/2x^2$.

Another approach is to enumerate in rectangular shells, with the following traversal function:

$$\max(x, y) = n$$

First, x will range from 0 to n while y remains at n . Then, y will range from 0 to $n-1$ while x stays fixed at n . The reason for y stopping one short is to not repeat the point $\langle n, n \rangle$. The shell iteration also requires t^2 number of single pair steps.

And it must be this way because the area of the square that results from adding multiple shells is size x^2 .

3.2 Index Tree

In a sense an index tree also has two free variables, that of depth and breadth, say d and b . There is a difference however. On a grid, each successive x value exposes one unbounded y sequence. For a tree where ever node has an unbounded number of children, each successive d value exposes an unbounded number of unbounded sequences. This is because each of an unbounded number of children have an unbounded number of grand children. Thus, while enumerated diagonals or shells of on a grid, the area covered, as measured by the number of points visited, grows with the square of the diagonal or shell count. In contrast the area covered by an Index Tree grows exponentially with count of diagonals included. The first difference of a quadratic is a constant value, so on a grid each diagonal grows in length by a constant. The first difference of an exponential is $e^{n+1} - e^n = e^n(e - 1)$ Hence, for the Index Tree each successive diagonal grows in length exponentially.

A diagonal traversal for enumerating the labels of the index tree step by step has been implemented in Java and is available at https://github.com/Thomas-Walker-Lynch/Ariadne/tree/examples_1.0. Here are the first four diagonals.

Step	Value
0	[]
1	[0]
2	[0,0], [1]
3	[0,0,0], [1,0], [0,1], [2]
4	[0,0,0,0], [1,0,0], [0,1,0], [2,0], [0,0,1], [1,1], [0,2], [3]

Table 3: Diagonal Enumeration of Index Labels

For this enumeration two variables are kept. One is the diagonal itself, the other is a list of 'tape machines'. A tape machine is an unbounded list with an index pointer. In the initialization step, step 0, the diagonal is set to the root node. Each successive step is taken by doing two things: A) descending from each node on the current diagonal, B) taking a breadth step from each node child node. This is similar to walking a diagonal on a grid in that it goes down and over one place; however, here the 'over' affects all the nodes thus far visited, rather than a single point.

We can see this down and over occurring in the table above. For A), each new diagonal appends a 0 to the sequences of the prior step and includes those. This appending of a zero is a downward step in the tree, stepping from those nodes to

their leftmost child. Hence, in step 1, due to A), the root node $[]$ becomes $[0]$. Similarly in looking at step 2 to step 3, the nodes listed on the step 2 diagonal, $[0, 0], [1]$ become $[0, 0, 0], [1, 0]$

For the B) part of the step, each child that has been visited is replaced with its right neighbor sibling. Step 1 has the first child visited, namely $[0]$. If you then look down the steps on the far right you will see the sequence $[0], [1], [2], \dots$ as each successive sibling is visited. The step 2 diagonal has two nodes that were children, namely $[0, 0], [1]$. At step 3 you can see that both of their right siblings are present, as $[0, 0] \rightarrow [0, 1]$ and $[1] \rightarrow [2]$.

At each step, due to A) incremental progress in depth is made at every step, and because of B) incremental progress in breadth for every child list is made. Consequently, given a sufficient large step bound, this enumeration algorithm will visit every node in the tree, and hence it will visit every possible bounded sequence of Natural Numbers. If the algorithm is stepped without bound, it will yield sequences without bound.

The enumeration of a two dimensional grid yields diagonals or shells that grow by a fixed increment per step. The enumeration of the Index Tree node labels yields a diagonal that grows exponentially per step. Though the diagonal grows much faster in length than for a grid, it is still being enumerated.

4 Cantor's Diagonal Derived Exception Algorithm in General Form

We can avoid number representation issues altogether by a more abstract formulation, that of functions over Natural Numbers. By 'over' we mean to say that each function accepts a Natural Number, and returns a Natural Number. Famously Cantor's method shows that any enumeration of the functions over Natural Numbers will be incomplete. In this section we will attempt to reproduce this result

Initially there is a proposed enumeration of the functions of natural numbers:

$F_0(0)$	$F_0(1)$	$F_0(2)$	$F_0(3)$	$F_0(4)$	$F_0(5)$	$\dots$
$F_1(0)$	$F_1(1)$	$F_1(2)$	$F_1(3)$	$F_1(4)$	$F_1(5)$	
$F_2(0)$	$F_2(1)$	$F_2(2)$	$F_2(3)$	$F_2(4)$	$F_2(5)$	
$F_3(0)$	$F_3(1)$	$F_3(2)$	$F_3(3)$	$F_3(4)$	$F_3(5)$	
$F_4(0)$	$F_4(1)$	$F_4(2)$	$F_4(3)$	$F_4(4)$	$F_4(5)$	
$F_5(0)$	$F_5(1)$	$F_5(2)$	$F_5(3)$	$F_5(4)$	$F_5(5)$	
$F_6(0)$	$F_6(1)$	$F_6(2)$	$F_6(3)$	$F_6(4)$	$F_6(5)$	$F_6(6)$
$F_7(0)$	$F_7(1)$	$F_7(2)$	$F_7(3)$	$F_7(4)$	$F_7(5)$	$F_7(7)$
$\vdots$						
$\vdots$						
$\vdots$						

Table 4: Functions of Natural Numbers

This is a table of Natural Numbers. For example, $F_0(0)$ is the Natural Number returned by F_0 when it is passed the Natural Number of 0. $F_0(1)$ is the Natural Number returned by the function F_0 when passed the Natural Number 1 etc. Hence, the columns enumerate the Natural Numbers, while each row represents a function of the Natural Numbers. As no definition has been given for the functions, the table is an abstraction for any enumeration of functions over the Natural Numbers.

In the table of the prior section, we each entry in the table was a symbol among the alphabet of binary digit symbols, namely 0,1. For this table we will speak more generally of having a Natural Number per table entry. Hence we generalize our symbol altering function to be a Natural Number altering function. And as promised earlier we will allow for some arithmetic, namely we will allow the operation of Natural Number addition. Accordingly, we can add any Natural Number other than zero to $F_i(j)$ to create a new function that has a different value at this point. Likewise, if $F_i(j)$ is nonzero, we can add any Natural Number that is less than $F_i(j)$ to create a new function that is different than $F_i(j)$.

Suppose for a Natural Number modifying function we chose +1, then the prefixes to the Exception function generated by Cantor's Algorithm at each step are:

$F_0(0) + 1$					
$F_0(0) + 1$	$F_1(1) + 1$				
$F_0(0) + 1$	$F_1(1) + 1$	$F_2(2) + 1$			
$F_0(0) + 1$	$F_1(1) + 1$	$F_2(2) + 1$	$F_3(3) + 1$		
$F_0(0) + 1$	$F_1(1) + 1$	$F_2(2) + 1$	$F_3(3) + 1$	$F_4(4) + 1$	
.					
.					
.					

Table 5: Prefixes of the Exception Function

The first prefix of the Exception Function differs from the row 0 function, F_0 for the domain value of 0, but we can not determine whether this function occurs later in the table since the functions F_n are unspecified. The next Exception Function prefix differs from the row 1 function, F_1 for the domain value of 1, and still differs from 0 function for the domain value of 0, but we are no closer to knowing if this function appears lower in the table, as will be the case for all the prefixes, because none of these help us know the definitions of the unspecified F_n functions further on in the table. Each time we determine an exception it simply pushes the question if it matches other functions further down the table.

Perhaps specific proposal for an enumeration of the functions over Natural Numbers would better help us understand Cantor's technique. As it so happens, we already have such a proposal in the previous section, because a sequence of Natural Numbers is a function of the sequence index, staring at index 0. So for example, in row 4 of the Diagonal Enumeration of Index Labels, we see the sequence [010] This is the function $F(0) = 0, F(1) = 1, F(2) = 0$. It follows, since the Index Tree iterator visits every node in the Index Tree, that it also enumerates every sequence of Natural Numbers, and thus enumerates every function of Natural Numbers.

	○			
0		○		
0	0		○	
1				○
0	0	0		
1	0			
0	1			
2				
0	0	0	0	
1	0	0		
0	1	0		
2	0			
0	0	1		
1	1			
0	2			
3				
.				
.				
.				

Table 6: Cantor Table for the Enumeration of Index Tree Sequences

Here the row 0 function has no domain, nor range. It is the null function. The row 1 function has only 0 in its domain, and the function defined at that point is 0. Looking further down the table for another example, row 10 is defined for the domain 0, 1, 2 with $F(0) = 0$, $F(1) = 1$, and $F(2) = 0$. Coverage of the domain indeed grows very slowly, but any bound chosen for the domain will eventually be passed. With an unbounded table, the domain is unbounded. Furthermore as every node in the Index Tree is eventually traversed, for an unbounded function, all single argument Functions over the Natural Numbers will be found in the table.

We find a similar situation as for the enumeration of Natural Numbers in that the diagonal grows faster than the row sequences. Each diagonal step begins with a descent from the prior diagonal, where this descent increases the length of the member sequences in the new diagonal by one. Also each diagonal grows exponentially in length, so each successive diagonal appears exponentially further down the enumeration of sequences table. This means the maximum sequence length is growing logarithmically. When we enumerated the Natural Numbers this same disparity in growth caused the diagonal to fill with zeros, here the diagonal will fill with null values while it ventures beyond the region for which the function domain is defined. Undefined areas are shown as blank in above table.

In this scenario, Cantor's exception generating algorithm requires choosing only one among all the possible digit altering functions So let us propose this one: If the domain value is not accepted by the row function, then the exception function

becomes 0 for that domain value. If the domain value is accepted by the row function, then the exception function value for that domain value will be 1 plus the row function value. In this manner the exception function value will differ from each row n function, F_n for the domain value of n . Below is a table of the resulting exception function prefixes.

0
0 0
0 0 0
0 0 0 0
0 0 0 0 0
.
.
.

Table 7: Prefixes of the Exception Function (Zero Function)

Note, the row 0 exception function prefix is found in row 1 of the table. The row 1 exception function prefix is found in row 2 of the table. The row 2 prefix is found in row 4. In general the n th prefix is found in row 2^n . Hence the diagonal method fails to create an exception to this enumeration of the functions over the Natural Numbers.